FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO

DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Criação de Camarão em Cativeiro com Monitoramento IoT para Gastronomia

Davi Mathais de Almeida
Leandro Augusto De Souza Muniz
Tiago de Lara Rodrigues

Registro – SP
2026

Índice

1 CANVAS: Modelo de Negócio e Estratégia **3**
1.1 Análise do Modelo de Negócio . . . 3

2 SWOT: Análise Estratégica **4**
2.1 Recomendações Estratégicas . . . 5

3 Website: Estrutura e Especificações Técnicas Web **6**
3.1 Mapa do Site da Equipe . . . 6
3.2 Conteúdo das Páginas . . . 7
3.3 Especificações Técnicas do Site da Equipe . . . 7

4 Diagramas UML: Modelagem Visual do Sistema **7**
4.1 Diagrama de Casos de Uso . . . 8
4.2 Diagrama de Classes . . . 9
4.3 Diagrama de Objetos . . . 10
4.4 Diagrama de Sequência . . . 10
4.5 Diagrama de Atividades . . . 11
4.6 Diagrama de Componentes . . . 13

5 Infraestrutura de Rede **13**
5.1 Diagrama de Implantação . . . 14
5.2 Diagrama de Estados . . . 14

6 Diagrama de Banco de Dados **15**
6.1 Modelo Conceitual . . . 15
6.2 Modelo Lógico . . . 16
6.3 Modelo Físico . . . 17

7 DevOps: Infraestrutura, Automação e Entrega Contínua **18**
7.1 Pipeline CI/CD . . . 18
7.2 Configuração CI/CD . . . 19
7.3 Ferramentas e Infraestrutura como Código . . . 26
7.4 Infraestrutura do Sistema . . . 26
7.4.1 Arquitetura de Rede . . . 27
7.4.2 Diagrama de Infraestrutura . . . 27
7.5 Arquitetura de Deploy . . . 28
7.6 Configuração Docker Compose (Desenvolvimento) . . . 29
7.7 Monitoramento e Alertas . . . 29
7.8 Métricas e SLA . . . 30

Referências **32**

Lista de Artefatos e suas Finalidades

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Diagramas UML:** Modelagem visual do sistema
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua

---

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas é uma ferramenta visual estratégica que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.

Tabela 1: Business Model Canvas - Visão Geral

<table>
<tr><td>Parcerias Principais</td><td>Atividades Chave</td><td>Proposta de Valor</td><td>Relacionamento</td><td>Segmentos</td></tr>
<tr><td>Fornecedores de sensores e microcontroladores (ESP32), provedores de infraestrutura em nuvem (Vercel, Render), consultores técnicos em carcinicultura</td><td>Coleta e interpretação de dados de temperatura, pH e amônia; geração de alertas em tempo real; manutenção da plataforma e dos sensores</td><td>Redução do tempo de manejo manual (de cerca de 5h/dia para monitoramento automatizado), alertas imediatos, maior precisão no controle da qualidade da água</td><td>Suporte técnico contínuo, treinamentos de uso, canal direto via chat/WhatsApp com a empresa</td><td>Pequenos e médios produtores de camarão e cooperativas de carcinicultura</td></tr>
<tr><td></td><td>Recursos Principais</td><td></td><td>Canais</td><td></td></tr>
<tr><td></td><td>Sensores (DS18B20, Ph4502, MQ-135), microcontrolador ESP32, plataforma web/PWA (React/Next.js, Node/Express, MongoDB Atlas), dispensador automatizado de ração</td><td></td><td>Indicação por consultores e associações do setor, feiras e eventos de aquicultura, redes sociais e e-mail para captação, telefone/chat para suporte pós-venda</td><td></td></tr>
<tr><td colspan="2">Estrutura de Custos</td><td colspan="3">Fontes de Receita</td></tr>
<tr><td colspan="2">Fixos: desenvolvimento e manutenção do sistema, hospedagem em nuvem (Vercel/Render). Variáveis: equipamentos IoT por instalação, marketing e aquisição de clientes</td><td colspan="3">Assinatura mensal ou anual (SaaS), taxa de instalação do sistema físico, treinamentos avulsos</td></tr>
</table>

1.1 Análise do Modelo de Negócio

A proposta de valor se apoia em resultados mensuráveis do próprio protótipo, como tempo de resposta a alertas imediata e precisão validada dos sensores, o que reduz o risco de a proposta soar genérica frente à concorrência. Ainda assim, o modelo depende de uma premissa não testada: que produtores tradicionalmente pouco digitalizados aceitem pagar recorrentemente (assinatura) por um serviço cuja alternativa (manejo manual) tem custo percebido como zero, já que é mão de obra própria. Essa é a principal tensão do Canvas e deveria ser endereçada com mais clareza, por exemplo, comparando o custo da mão de obra desviada para outras tarefas

com o valor da assinatura.

A separação entre parcerias e segmentos evita um erro comum: tratar o próprio cliente (produtor de camarão) como parceiro estratégico. Os parceiros reais são quem viabiliza a operação sem ser quem paga: fornecedores de hardware e provedores de nuvem, dos quais o negócio depende diretamente para funcionar, e consultores técnicos, que também aparecem como canal de aquisição por indicação, conectando os blocos de parceria e canal de forma coerente.

A estrutura de custos evidencia um ponto de atenção para a sustentabilidade do modelo de assinatura: os custos variáveis por instalação (sensores, microcontrolador) são custos únicos, mas o modelo de receita é recorrente, ou seja, a margem depende de reter o cliente por tempo suficiente para diluir o custo do hardware inicial, o que reforça a importância do bloco de relacionamento (suporte e treinamento) como mecanismo de retenção, não apenas de satisfação.

2 SWOT: Análise Estratégica

A Análise SWOT (*Strengths, Weaknesses, Opportunities, Threats*), ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) (CHIAVENATO; SAPIRO, 2020), é uma ferramenta de planejamento estratégico que identifica os fatores internos (forças e fraquezas) e externos (oportunidades e ameaças) que impactam o projeto.

Tabela 2: Análise SWOT do Projeto

FATORES INTERNOS	
FORÇAS (Strengths)	**FRAQUEZAS (Weaknesses)**
• Sistema integrado que reúne monitoramento da água, alertas e programação remota da alimentação em uma única plataforma • Alertas gerados a cada leitura recebida (leituras a cada minuto), por notificação *push* e e-mail • Aplicação web progressiva, acessível por celular e computador sem lojas de aplicativos, com perfis de acesso (proprietário e funcionários) • Arquitetura baseada em componentes comerciais acessíveis (ESP32 e sensores de uso geral) • Desenvolvimento orientado por relatos de produtores sobre perdas por falha de alimentação e por alteração dos parâmetros da água	• Monitoramento de amônia não validado: o sensor utilizado detecta gás no ar, e não amônia dissolvida • Dependência de Wi-Fi e da API: o firmware não armazena leituras durante quedas de conexão • Validação restrita a um aquário, uma espécie e cerca de 5,5 horas, sem validação com produtor ou cliente pagante • Desvio sistemático no pH, com ajuste em ponto único e referência de baixo custo • Dependência de componentes importados (ESP32, sensores), sujeitos a variação cambial e prazo de importação
FATORES EXTERNOS	
OPORTUNIDADES (Opportunities)	**AMEAÇAS (Threats)**
• Crescimento do mercado de aquicultura e da demanda por rastreabilidade/sustentabilidade na produção • Parcerias com instituições de pesquisa (ex: universidades) para validação científica e captação de fomento • Expansão do sistema para outros tipos de monitoramento ambiental agrícola • Linhas de crédito e programas de incentivo à automação rural	• Cobertura *Wi-Fi* limitada em propriedades rurais do Vale do Ribeira, exigindo soluções de conectividade alternativas • Surtos de doenças no cultivo (ex: mancha branca), que afetam a demanda por qualquer tecnologia de manejo, independente de sua eficácia • Volatilidade do preço do camarão no mercado, reduzindo a margem do produtor para investir em novas tecnologias • Entrada de concorrentes com soluções similares e maior capital para escala comercial

2.1 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Utilizar os resultados validados (tempo de resposta, precisão dos sensores) como diferencial competitivo em parcerias com instituições de pesquisa,

fortalecendo a credibilidade técnica do produto frente a linhas de fomento e programas de incentivo à automação rural

- **Forças + Ameaças:** Aproveitar a arquitetura de componentes acessíveis e a aplicação web sem lojas de aplicativos para reduzir a barreira de entrada de produtores de menor capital e, para mitigar a limitação de conectividade, priorizar o armazenamento local de leituras no ESP32 e investigar conectividade LoRa/4G
- **Fraquezas + Oportunidades:** Buscar validação com cliente real via parceria com instituição de pesquisa ou programa de incentivo, transformando o piloto acadêmico em piloto comercial monitorado, o que também endereça a falta de dados de mercado
- **Fraquezas + Ameaças:** Implementar plano de manutenção preventiva dos sensores e redundância na integração de hardware para reduzir o risco de falhas técnicas, especialmente relevante diante da entrada de concorrentes com maior capital, falhas recorrentes seriam o motivo mais fácil de perder espaço de mercado

Tabela 3: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#146AC7	Botões principais, links e destaques
Secundária		#FFC2C2	Identidade visual e destaques
Fundo		#FFFFFF	*Background* padrão
Fundo Informações		#F8FAFC	*Background* de informações
Texto		#000000	Texto principal
Sucesso		#166534	Mensagens de sucesso
Erro		#991B1B	Mensagens de erro

O azul primário apresenta contraste de cerca de 5,4:1 com o branco, e as cores de sucesso e de erro superam 7:1, acima do mínimo de 4,5:1 recomendado pela WCAG para texto.

Tabela 4: Tipografia do Projeto

Elemento	Fonte	Tamanho	Uso
Títulos	Poppins Bold	19–28 px	Cabeçalhos de seções e páginas
Corpo	Poppins Regular/Medium	13–16 px	Texto principal e descrições
Botões	Poppins Medium	13–14 px	Botões de ação e navegação

3 Website: Estrutura e Especificações Técnicas Web

Esta seção documenta o website institucional/portfólio da equipe que desenvolveu a aplicação. O site da equipe serve para apresentar os membros, projetos realizados, competências e informações de contato.

3.1 Mapa do Site da Equipe

O mapa do site (*sitemap*) mostra a hierarquia e organização das páginas do website da equipe:

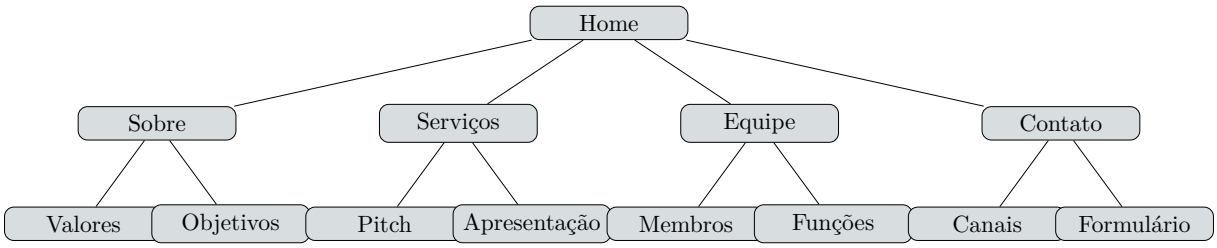


Figura 1: Estrutura de navegação do website da equipe

3.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** Apresentação inicial da equipe
- **Sobre:** Conteúdo sobre nossos valores, missão e visão
- **Serviços:** Descrição dos serviços oferecidos pela equipe
- **Equipe:** Perfil dos membros (nome, foto, função, LinkedIn/GitHub)
- **Contato:** Formulário de contato e links para redes sociais

3.3 Especificações Técnicas do Site da Equipe

Tabela 5: Stack Tecnológica do Website da Equipe

Camada	**Tecnologia**	**Descrição**
Frontend	React 18	Biblioteca JavaScript para interfaces
Build Tool e Bundler	Vite	Ferramenta de build de altíssima performance e dev server
Linguagem	TypeScript	Superset do JavaScript que garante tipagem estática segura
Estilização e Design	Tailwind CSS v4	Framework CSS utilitário para responsividade e temas
Motor de Animações	Framer Motion	Biblioteca declarativa para animações e transições complexas
Iconografia Vetorial	Lucide / React Icons	Bibliotecas de ícones modernos, responsivos e otimizados
Serviço de Mensageria	EmailJS	Plataforma *backend-as-a-service* para envio de e-mails
Hospedagem e *Deploy*	Vercel	Plataforma de *Edge Network* e *hosting* com *deploy* contínuo

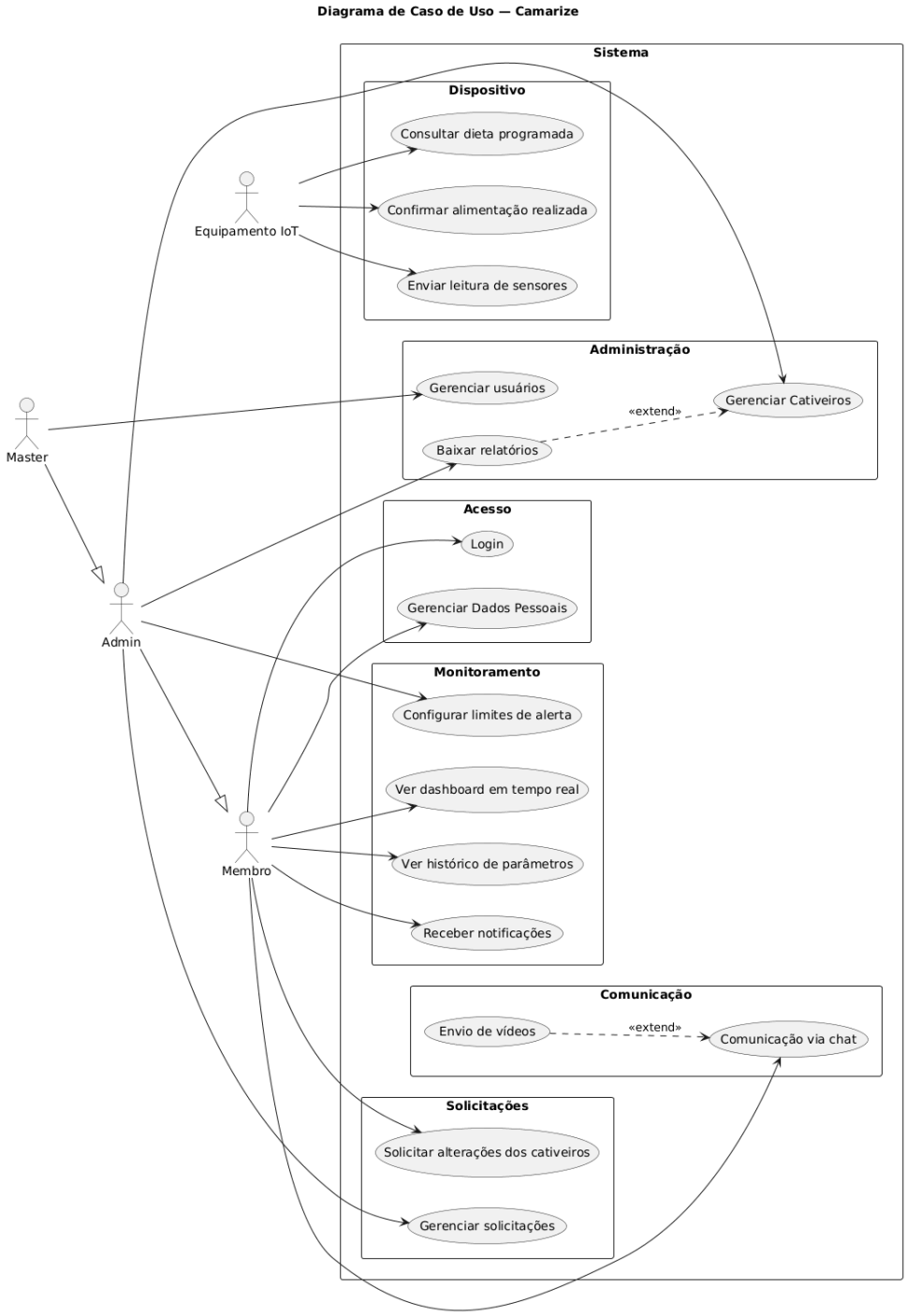
4 Diagramas UML: Modelagem Visual do Sistema

A Unified Modeling Language (UML) (BOOCH; RUMBAUGH; JACOBSON, 2005) é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de *software*.

4.1 Diagrama de Casos de Uso

O diagrama identifica quatro atores principais e suas interações com o sistema: o Equipamento IoT, o Funcionário, o Dono e a Equipe técnica, cada um com permissões e funcionalidades específicas.

Figura 2: Diagrama de Casos de Uso

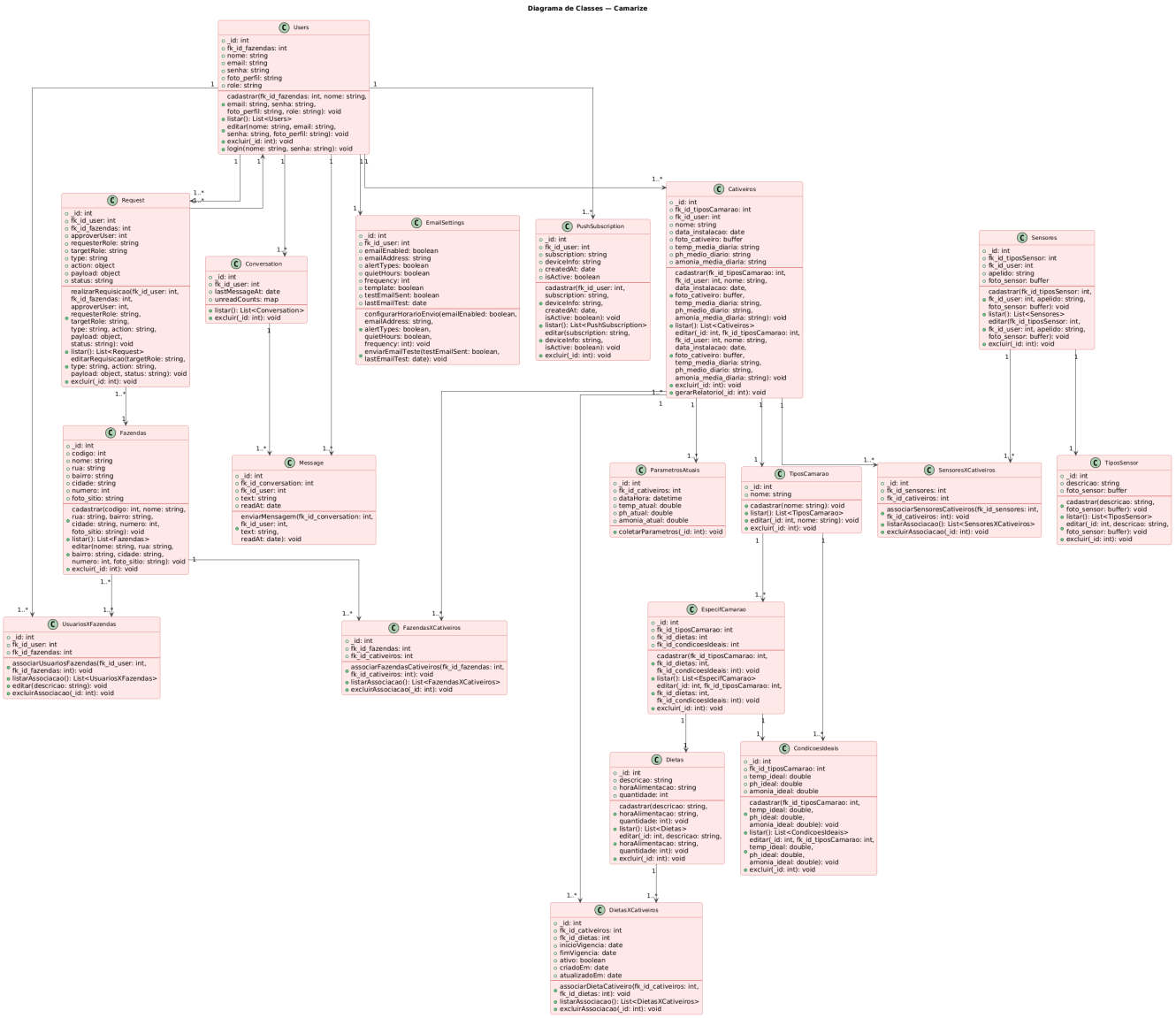


Fonte: Autoria própria

4.2 Diagrama de Classes

Apresenta a estrutura orientada a objetos do sistema, com Users como classe central e seus relacionamentos com Cativeiros, Fazendas e os módulos de comunicação e notificações. Inclui os métodos de cada classe e as associações N:N por meio de coleções intermediárias.

Figura 3: Diagrama de Classes

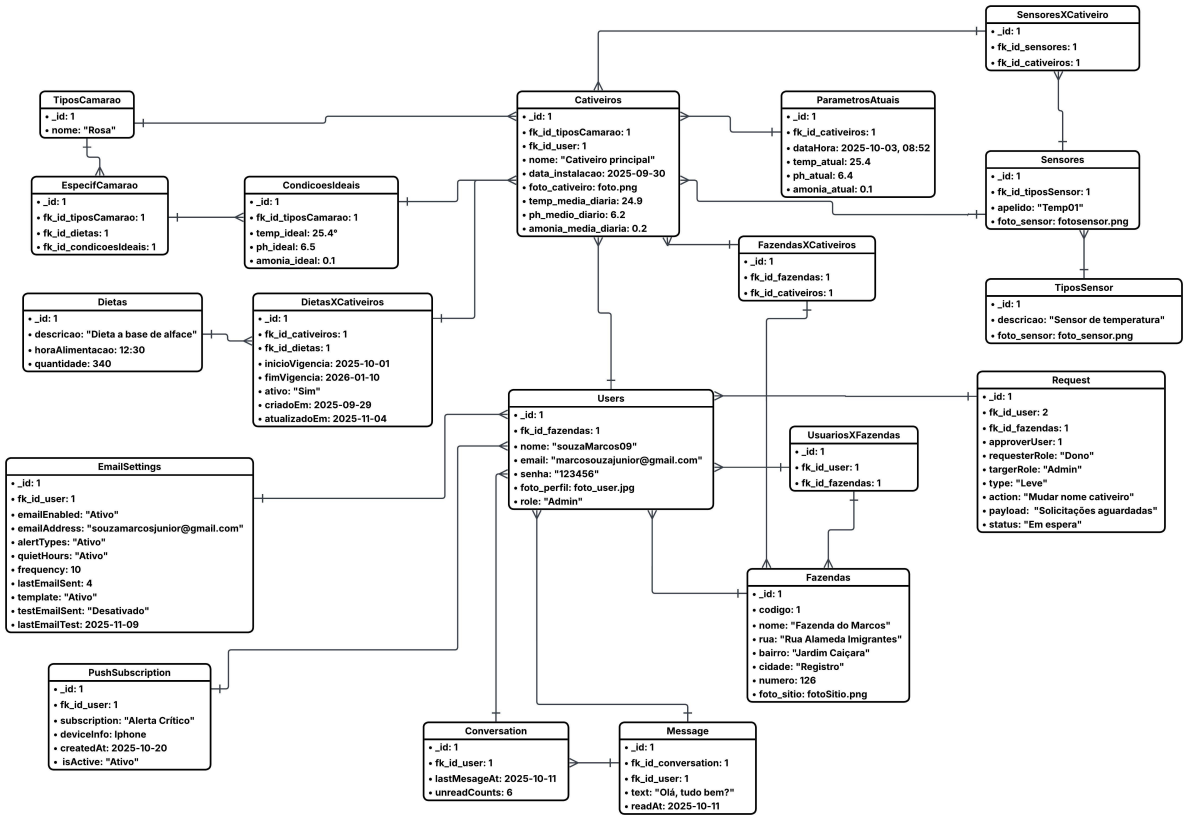


Fonte: Autoria própria

4.3 Diagrama de Objetos

O diagrama de objetos ilustra instâncias concretas das classes do sistema em um cenário de uso, exemplificando os valores dos atributos e os vínculos entre objetos em tempo de execução.

Figura 4: Diagrama de Objetos

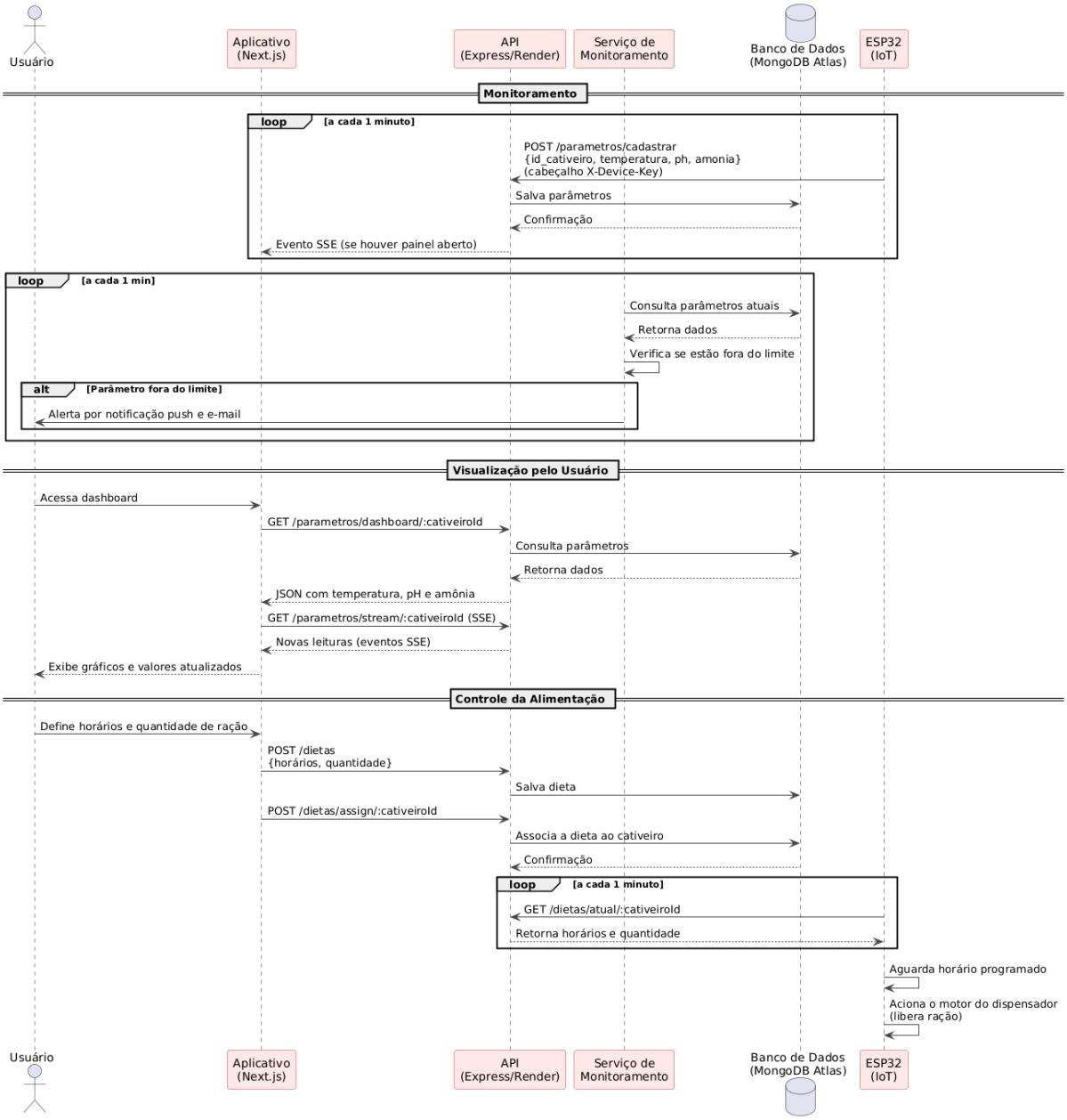


Fonte: Autoria própria

4.4 Diagrama de Sequência

Detalha três fluxos principais de interação: monitoramento em tempo real, visualização pelo usuário e controle de alimentação, envolvendo o Aplicativo, a API, o Banco de Dados e o ESP32.

Figura 5: Diagrama de Sequência

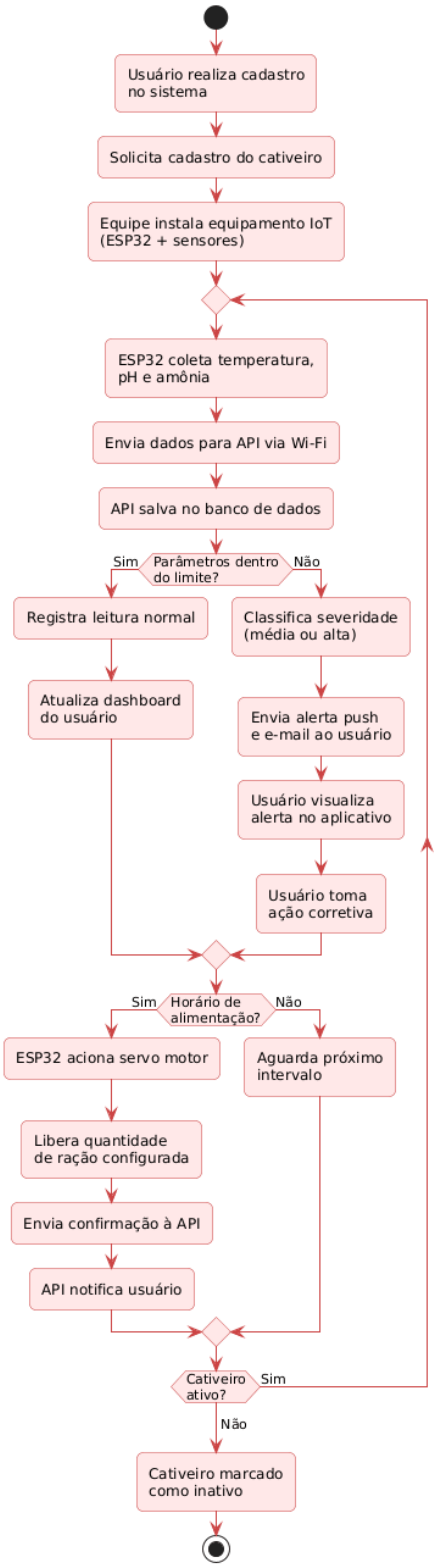


Fonte: Autoria própria

4.5 Diagrama de Atividades

Descreve o fluxo completo de operação do sistema, desde o cadastro do usuário até o ciclo contínuo de monitoramento e controle de alimentação, incluindo a lógica de verificação de limites e envio de alertas.

Figura 6: Diagrama de Atividades

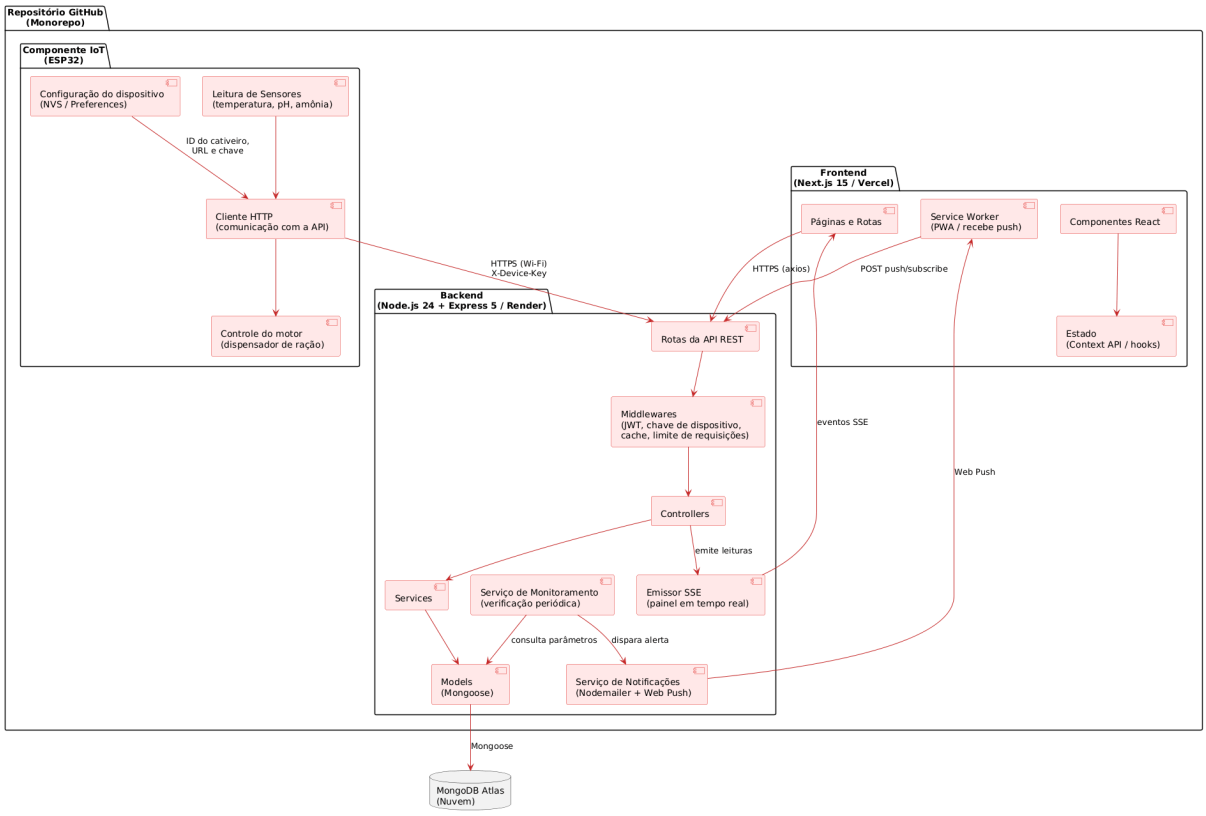


Fonte: Autoria própria

4.6 Diagrama de Componentes

Organiza o sistema em três blocos principais: Frontend (Next.js/Vercel), Backend (Node.js + Express/Render) e Componente IoT (ESP32), com comunicação via HTTPS e conexão ao MongoDB Atlas via Mongoose.

Figura 7: Diagrama de Componentes



Fonte: Autoria própria

5 Infraestrutura de Rede

A Figura 8 apresenta a infraestrutura de rede do sistema. No cativeiro, cada ESP32 DevKit V1, ao qual estão conectados os sensores de temperatura (DS18B20), pH (PH-4502C) e gases (MQ-135) e o motor de passo do dispensador de ração (28BYJ-48 com driver ULN2003), conecta-se por *Wi-Fi* (padrão 802.11 b/g/n, banda de 2,4 GHz) a um roteador local, que dá acesso à internet por meio do provedor. A cada minuto, o ESP32 envia as leituras à API, hospedada na Render, e, nos horários programados, consulta a ela a configuração de alimentação, ambas as comunicações por HTTPS (porta 443).

A API grava os dados no MongoDB Atlas, banco de dados em nuvem, por conexão com TLS, e avalia cada leitura ao recebê-la. Quando um limite configurado é ultrapassado, ela aciona os alertas por e-mail (Nodemailer, via SMTP do Gmail) e por notificação *push* (Web Push, protocolo VAPID). O frontend, desenvolvido em Next.js e disponibilizado como aplicação web progressiva, é hospedado na Vercel e acessado pelos usuários, em smartphone ou notebook, por HTTPS (porta 443), a partir de qualquer rede.

Como consequência dessa arquitetura, o funcionamento do sistema no cativeiro depende de conectividade *Wi-Fi* local e de acesso à internet: durante interrupções, as leituras não são enviadas e o ESP32 não consegue consultar a configuração de alimentação.

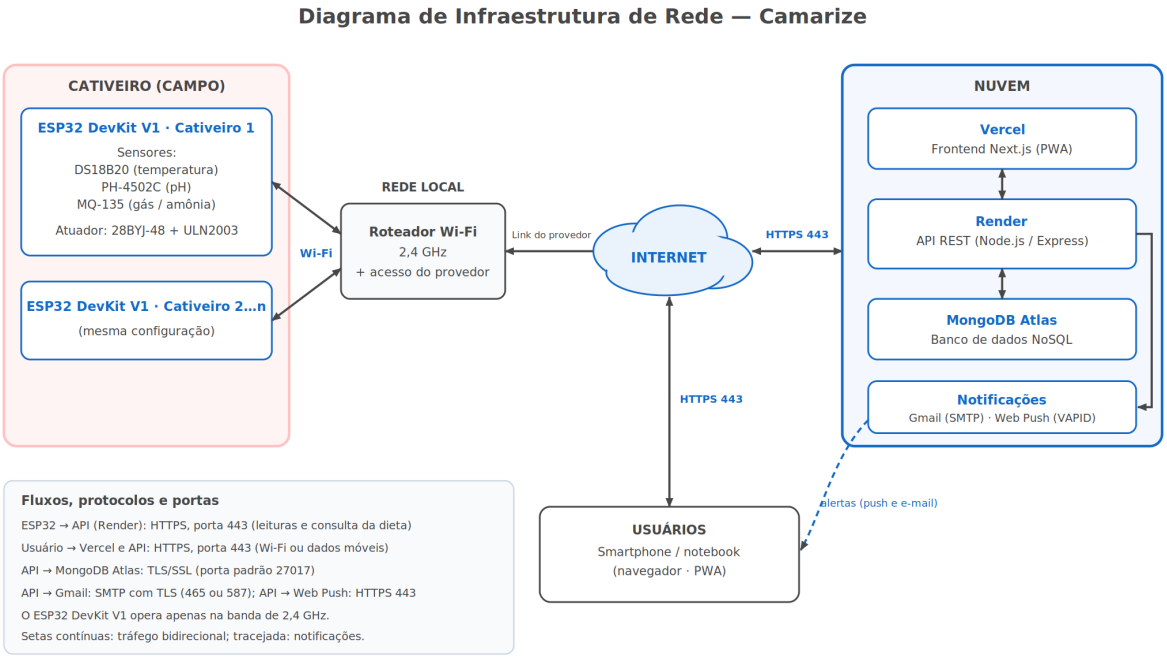
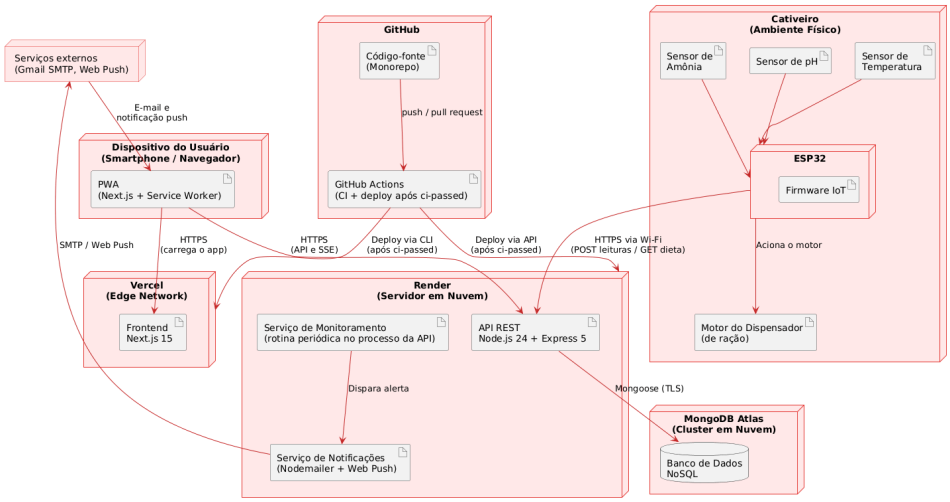


Figura 8: Diagrama de infraestrutura de rede do sistema

5.1 Diagrama de Implantação

Apresenta a distribuição física dos componentes em produção, incluindo a aplicação PWA na Vercel, a API REST no Render, o banco MongoDB Atlas e o ESP32 no ambiente do cativeiro, com *deploy* contínuo via GitHub.

Figura 9: Diagrama de Implantação

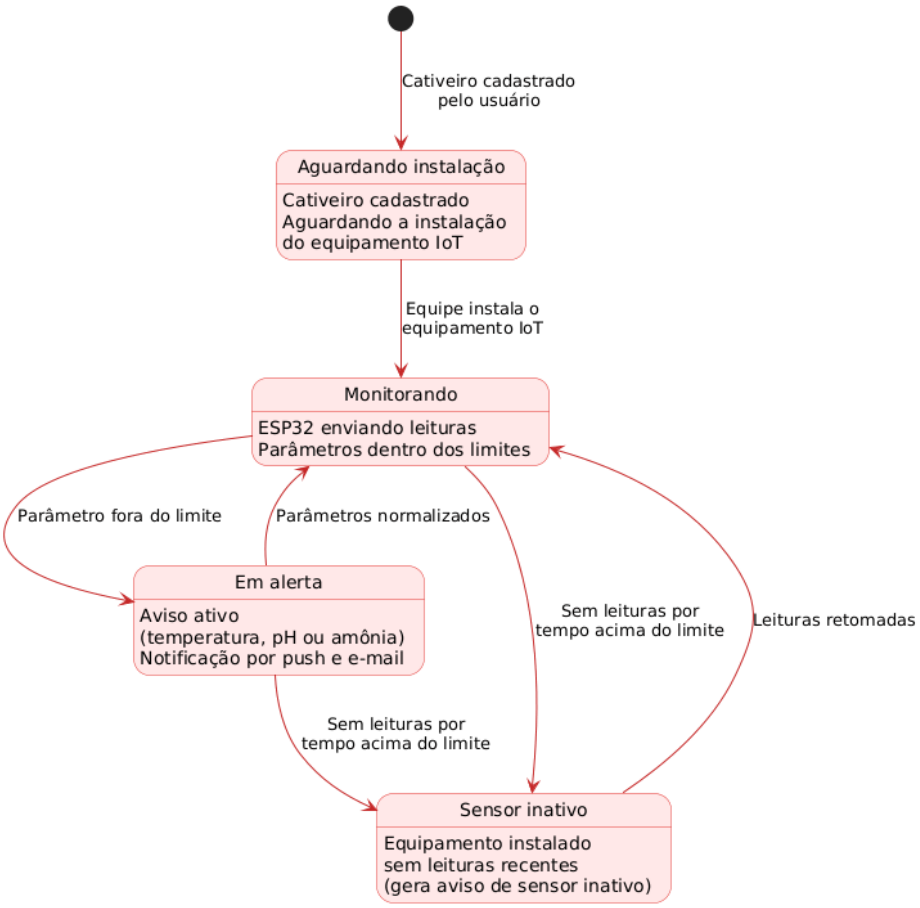


Fonte: Autoria própria

5.2 Diagrama de Estados

Modela o ciclo de vida de um cativeiro em quatro estados: AguardandoInstalação, Monitorando, EmAlerta e Inativo, com transições acionadas por eventos do sistema e ações do usuário.

Figura 10: Diagrama de Estados



Fonte: Autoria própria

Para realizar a montagem dos diagramas, foi utilizada a ferramenta *online* PlantUML, que permite a criação de diagramas UML através de código.

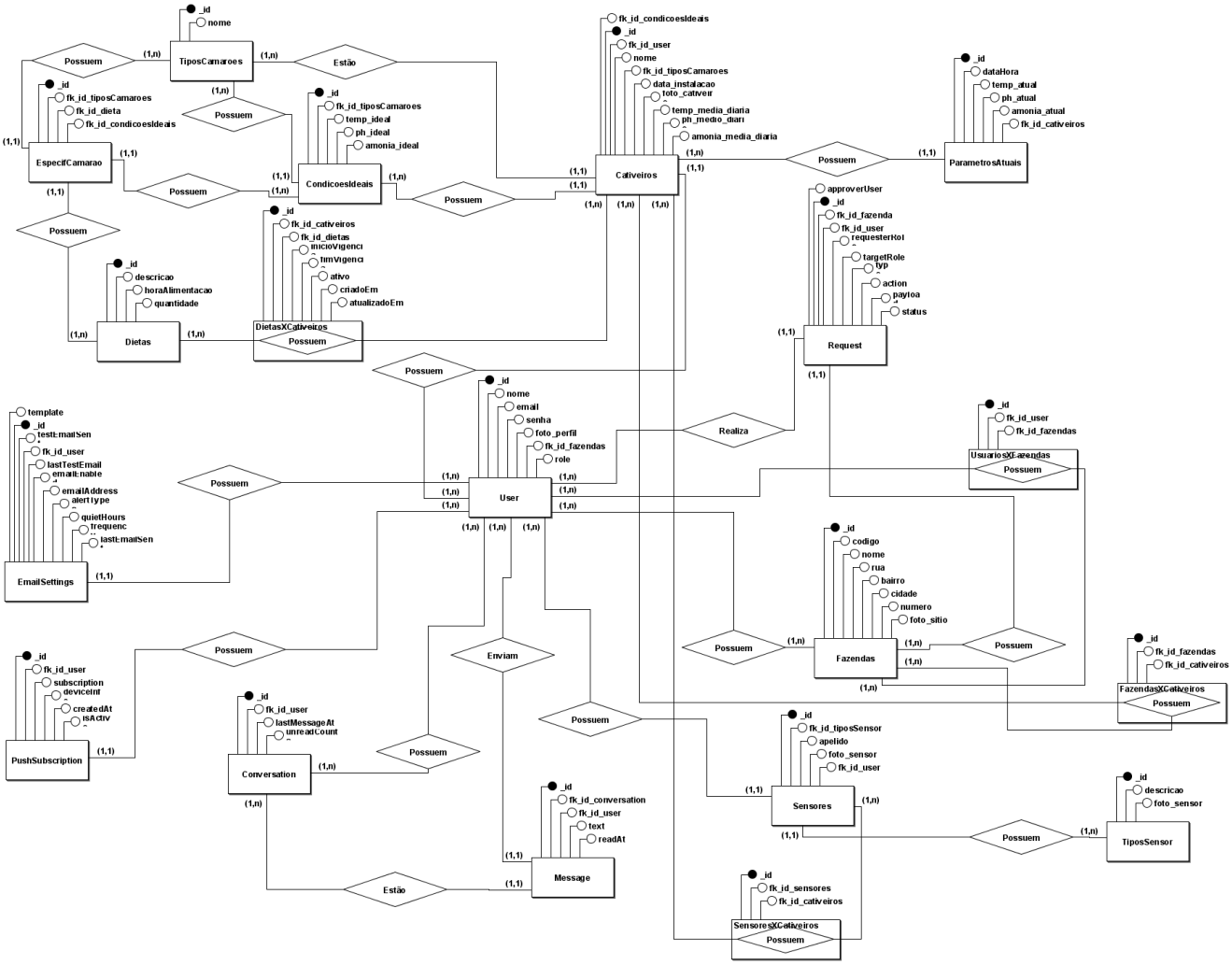
6 Diagrama de Banco de Dados

O diagrama de banco de dados representa a estrutura do banco de dados não relacional (NoSQL) utilizado no projeto, mostrando as coleções, seus campos e os relacionamentos entre elas.

6.1 Modelo Conceitual

O modelo conceitual representa as entidades e seus relacionamentos em alto nível, destacando as cardinalidades (1,1) e (1,N) entre todas as entidades do sistema de forma independente de tecnologia.

Figura 11: Modelo Conceitual do Banco de Dados

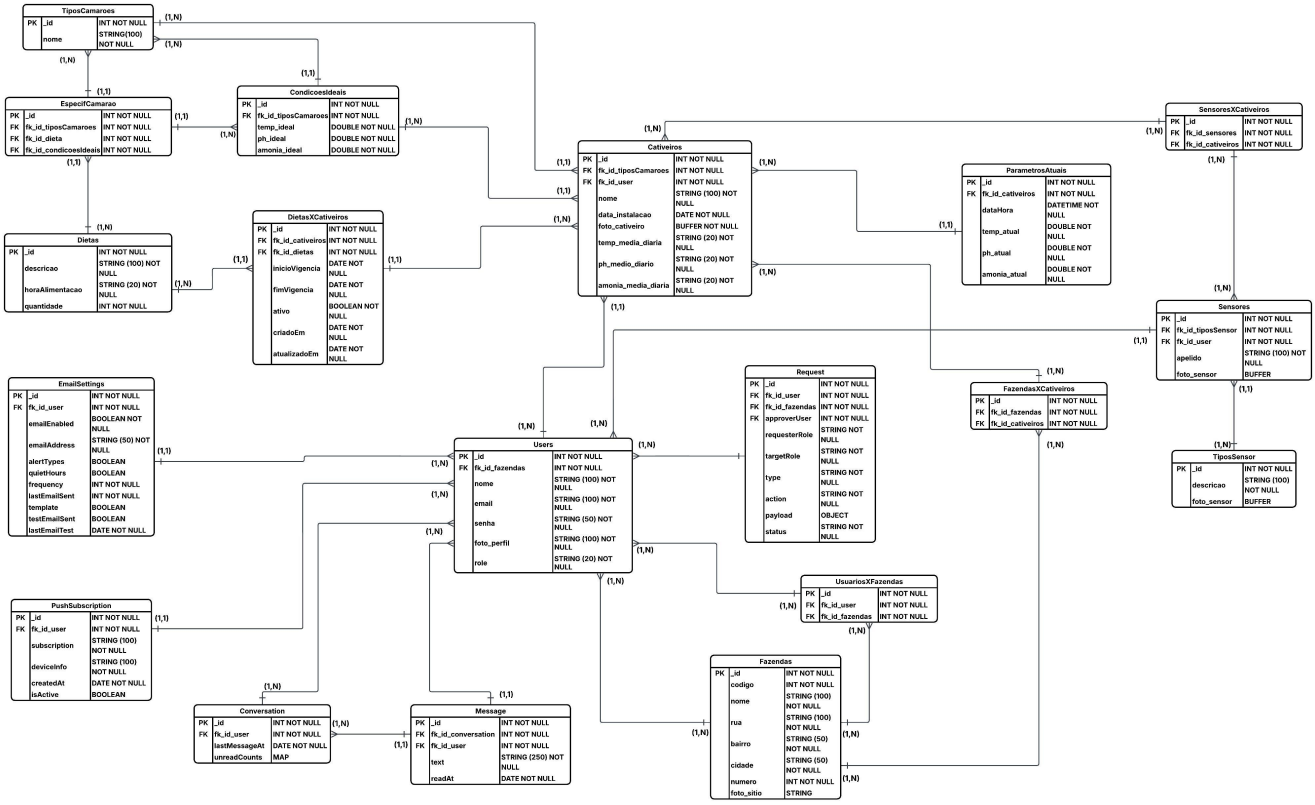


Fonte: Autoria própria

6.2 Modelo Lógico

O modelo lógico refina o conceitual para um esquema de coleções MongoDB, detalhando os documentos como Tipos_camarao, Cativeiros, Sitios, Parametros_atuais, Dietas, Sensores, Usuarios, entre outros, com seus campos, tipos de dados e referências entre coleções.

Figura 12: Modelo Lógico do Banco de Dados

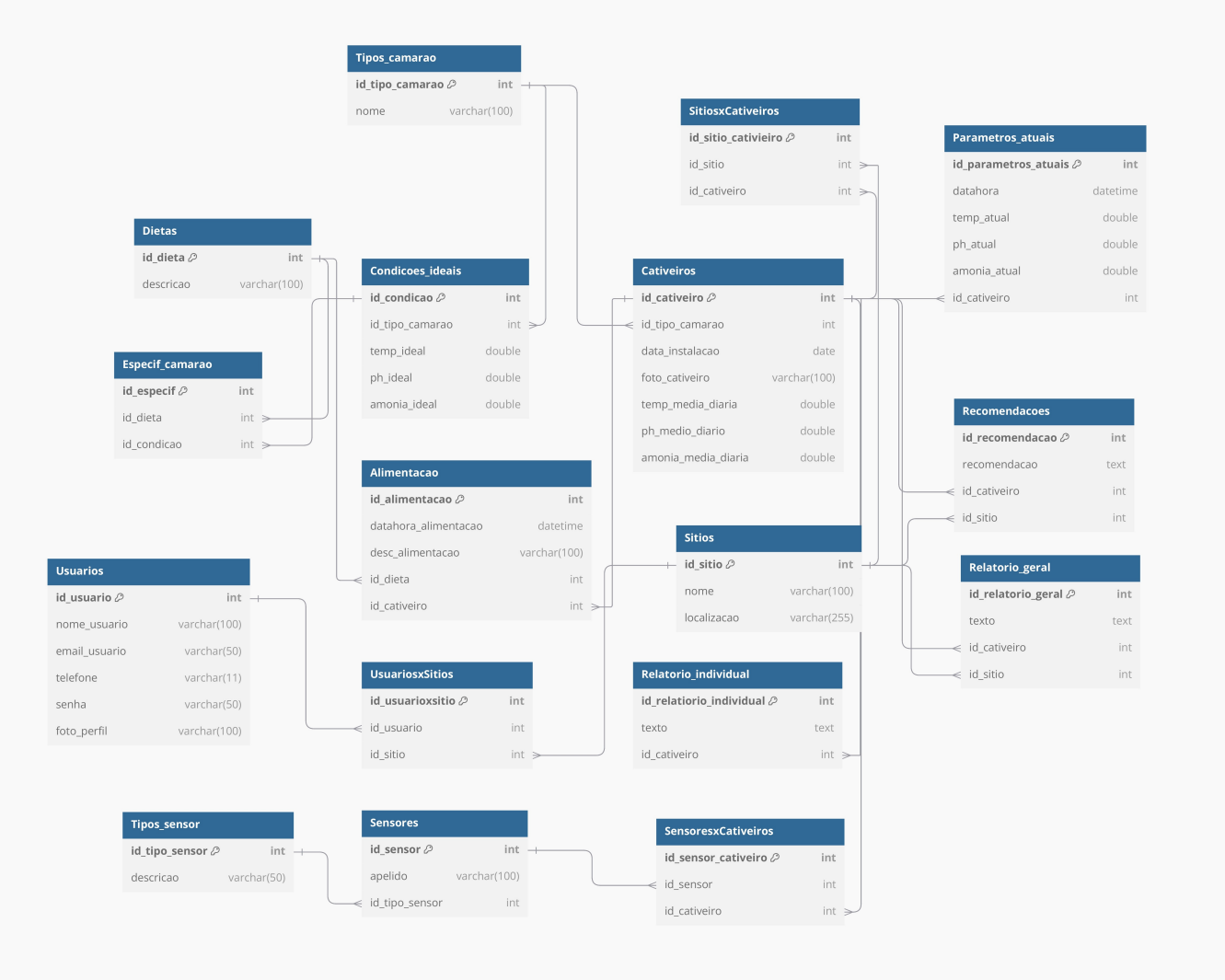


Fonte: Autoria própria

6.3 Modelo Físico

O modelo físico é a implementação final utilizando *Mongoose Schemas*, com definições de tipos (String, Number, Boolean, Buffer, Map) e restrições (`required: true`), voltado para a criação direta das coleções no MongoDB Atlas, contemplando todas as entidades do sistema incluindo EmailSettings, PushSubscription, Conversation, Message e Request.

Figura 13: Modelo Físico do Banco de Dados



Fonte: Autoria própria

7 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps (KIM et al., 2018) é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua.

7.1 Pipeline CI/CD

O pipeline (GitHub Actions) é executado a cada *push* e *pull request* na branch `main` e reúne: lint (ESLint) e verificação de sintaxe da API; testes unitários da API com cobertura, executados contra um MongoDB em contêiner; *build* de produção do frontend; verificação de segredos versionados; testes de ponta a ponta (Playwright); auditoria de dependências (*npm audit*); e *build* das imagens Docker. O job `ci-passed` consolida o resultado e, em *push* na `main`, somente quando ele é bem-sucedido, os jobs de deploy publicam o frontend na Vercel (via CLI) e a API no Render (via API da plataforma, com verificação de prontidão). O auto-deploy nativo das duas plataformas foi desativado para que nada seja publicado sem passar pelo CI. Há ainda um fluxo de publicação manual, com confirmação explícita, para republicar um commit específico.

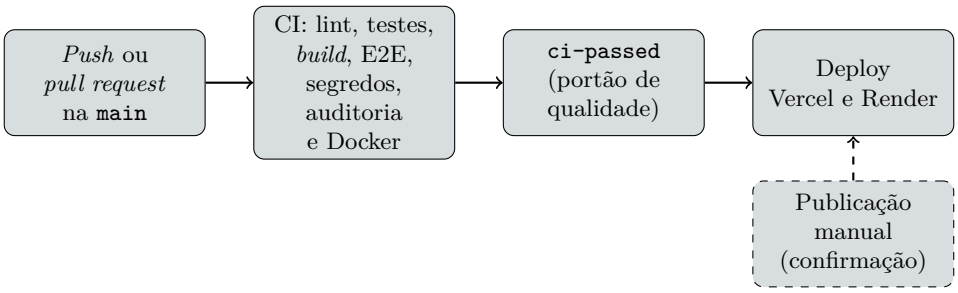


Figura 14: Pipeline CI/CD

7.2 Configuração CI/CD

O deploy depende do resultado do job `ci-passed`, que só reprova quando falham o lint, os testes da API, o *build* do frontend, a verificação de segredos ou os testes de ponta a ponta; a auditoria de dependências e o *build* das imagens Docker são informativos. O trecho a seguir (de `.github/workflows/ci.yml`) mostra a condição de publicação:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint-api:
    name: Lint API
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: api
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: api/package-lock.json

      - run: npm ci

      - name: Verificação de sintaxe (node --check)
        run: |
          find . -name "*.js" -not -path "./node_modules/*" \
            -not -path "./tests/*" -exec node --check {} \;
```

```
      - name: ESLint
        run: npm run lint

  test-api:
    name: Testes API
    runs-on: ubuntu-latest
    needs: lint-api
    services:
      mongo:
        image: mongo:7.0
        ports: ['27017:27017']
        options: >-
          --health-cmd "mongosh --quiet --eval 'db.adminCommand({ping:1})'"
          --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      MONGO_TEST_URL: mongodb://127.0.0.1:27017
    defaults:
      run:
        working-directory: api
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: api/package-lock.json

      - run: npm ci

      - name: Rodar testes unitários com cobertura
        run: npm run test:coverage

      - name: Upload relatório de cobertura
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: api-coverage-report
          path: api/coverage/
          retention-days: 14

  build-frontend:
    name: Build Frontend
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: front-react
    env:

      NEXT_PUBLIC_API_URL: http://localhost:4000
```

```
      NEXT_PUBLIC_SSE_URL: http://localhost:4000
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: front-react/package-lock.json

      - run: npm ci

      - name: Build de produção (Next.js)
        run: npm run build

  check-secrets:
    name: Verificação de Segredos
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20

      - name: Rodar scanner de segredos e dumps versionados
        run: node scripts/ci/check-secrets.mjs

  e2e:
    name: Testes E2E (Playwright)
    runs-on: ubuntu-latest
    needs: [lint-api, build-frontend]
    services:
      mongo:
        image: mongo:7.0
        ports: ['27017:27017']
        options: >-
          --health-cmd "mongosh --quiet --eval 'db.adminCommand({ping:1})'"
          --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      MONGO_TEST_URL: mongodb://127.0.0.1:27017
      NEXT_PUBLIC_API_URL: http://127.0.0.1:4100
      NEXT_PUBLIC_SSE_URL: http://127.0.0.1:4100
      NEXT_TELEMETRY_DISABLED: '1'
      REQUIRE_PUBLIC_ENV: 'true'
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
```

```
          cache: npm
          cache-dependency-path: |
            package-lock.json
            api/package-lock.json
            front-react/package-lock.json

      - run: npm ci
      - run: npm ci --prefix api
      - run: npm ci --prefix front-react

      - name: Build de produção (Next.js) com URLs de teste
        run: npm run build --prefix front-react

      - run: npx --no-install playwright install --with-deps chromium

      - name: Rodar testes E2E
        run: npm run test:e2e

      - name: Upload relatório de testes do navegador
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: browser-test-results
          path: |
            playwright-report/
            test-results/
          retention-days: 14

  audit:
    name: Auditoria de Dependências
    runs-on: ubuntu-latest
    strategy:
      matrix:
        directory: [api, front-react]
    defaults:
      run:
        working-directory: ${{ matrix.directory }}
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: ${{ matrix.directory }}/package-lock.json

      - run: npm ci

      - name: Auditoria npm (${{ matrix.directory }})
        run: npm audit --omit=dev --audit-level=critical
        continue-on-error: true
```

```
docker-build:
  name: Docker Build
  runs-on: ubuntu-latest
  needs: [test-api, build-frontend]
  steps:
    - uses: actions/checkout@v7

    - name: Build imagem API
      run: docker build -t camarize-api:ci ./api

    - name: Build imagem Frontend
      run: |
        docker build \
          --build-arg NEXT_PUBLIC_API_URL=http://localhost:4000 \
          --build-arg NEXT_PUBLIC_SSE_URL=http://localhost:4000 \
          -t camarize-frontend:ci ./front-react

ci-passed:
  name: CI Passou
  runs-on: ubuntu-latest
  if: always()
  needs:
    - lint-api
    - test-api
    - build-frontend
    - check-secrets
    - e2e
    - audit
    - docker-build
  steps:
    - name: Verificar resultados
      run: |
        echo "Lint API:            ${{ needs.lint-api.result }}"
        echo "Test API:            ${{ needs.test-api.result }}"
        echo "Build Frontend:      ${{ needs.build-frontend.result }}"
        echo "Verificação Segredos: ${{ needs.check-secrets.result }}"
        echo "E2E:                 ${{ needs.e2e.result }}"
        echo "Audit:               ${{ needs.audit.result }}"
        echo "Docker Build:        ${{ needs.docker-build.result }}"

        if [[ "${{ needs.lint-api.result }}" == "failure" ]] || \
           [[ "${{ needs.test-api.result }}" == "failure" ]] || \
           [[ "${{ needs.build-frontend.result }}" == "failure" ]] || \
           [[ "${{ needs.check-secrets.result }}" == "failure" ]] || \
           [[ "${{ needs.e2e.result }}" == "failure" ]]; then
          echo "CI falhou em jobs obrigatórios!"
          exit 1
        fi

        echo "Todos os jobs obrigatórios passaram!"
```

```
deploy-frontend:
  name: Deploy Frontend (Vercel)
  runs-on: ubuntu-latest
  needs: ci-passed
  if: >-
    github.event_name == 'push' &&
    github.ref == 'refs/heads/main' &&
    needs.ci-passed.result == 'success'
  environment: production
  env:
    VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
    VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
    VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
  steps:
    - uses: actions/checkout@v7

    - uses: actions/setup-node@v7
      with:
        node-version-file: .nvmrc

    - name: Conferir configuração da Vercel
      run: |
        test -n "$VERCEL_TOKEN" || \
          { echo "Configure o secret VERCEL_TOKEN."; exit 1; }
        test -n "$VERCEL_ORG_ID" || \
          { echo "Configure a variável VERCEL_ORG_ID."; exit 1; }
        test -n "$VERCEL_PROJECT_ID" || \
          { echo "Configure a variável VERCEL_PROJECT_ID."; exit 1; }

    - name: Pull do ambiente de produção da Vercel
      working-directory: front-react
      run: |
        npx --yes vercel@64.5.0 pull --yes \
          --environment=production --token="$VERCEL_TOKEN"

    - name: Build de produção (via Vercel)
      working-directory: front-react
      run: npx --yes vercel@64.5.0 build --prod --token="$VERCEL_TOKEN"

    - name: Deploy para produção
      working-directory: front-react
      run: |
        npx --yes vercel@64.5.0 deploy \
          --prebuilt --prod --token="$VERCEL_TOKEN"

deploy-api:
  name: Deploy API (Render)
  runs-on: ubuntu-latest
  needs: ci-passed
  if: >-
```

```
    github.event_name == 'push' &&
    github.ref == 'refs/heads/main' &&
    needs.ci-passed.result == 'success'
  environment: production
  env:
    RENDER_API_KEY: ${{ secrets.RENDER_API_KEY }}
    RENDER_SERVICE_ID: ${{ vars.RENDER_SERVICE_ID }}
    API_URL: ${{ vars.API_URL }}
  steps:
    - uses: actions/checkout@v7

    - uses: actions/setup-node@v7
      with:
        node-version-file: .nvmrc

    - name: Conferir configuração do Render
      run: |
        test -n "$RENDER_API_KEY" || \
          { echo "Configure o secret RENDER_API_KEY."; exit 1; }
        test -n "$RENDER_SERVICE_ID" || \
          { echo "Configure a variável RENDER_SERVICE_ID."; exit 1; }
        test -n "$API_URL" || \
          { echo "Configure a variável API_URL."; exit 1; }

    - name: Publicar no Render e aguardar readiness
      run: node scripts/ci/deploy-render.mjs
```

7.3 Ferramentas e Infraestrutura como Código

Tabela 6: Ferramentas DevOps Utilizadas

Categoria	**Ferramenta**	**Finalidade**
Controle de Versão	Git + GitHub	Versionamento de código e colaboração
CI/CD	GitHub Actions	Lint, testes, *build*, verificação de segredos, auditoria e deploy
Testes	Vitest + Playwright	Testes unitários da API e testes de ponta a ponta
Dependências	Dependabot	Atualizações semanais de dependências
Hosting Frontend	Vercel	Hospedagem do Next.js (deploy pelo GitHub Actions)
Hosting Backend	Render	Hospedagem da API Express em produção (deploy pelo GitHub Actions)
Containerização	Docker + Compose	Desenvolvimento local e validação das imagens no CI
Banco de Dados	MongoDB Atlas	Banco de dados NoSQL em nuvem (cluster gerenciado)
Documentação API	Swagger UI	Documentação interativa de endpoints REST

7.4 Infraestrutura do Sistema

Esta seção documenta como o sistema funciona em diferentes ambientes e plataformas.

7.4.1 Arquitetura de Rede

Tabela 7: Componentes de Infraestrutura

Componente	Descrição
Rede Local (LAN)	• Ambiente local para desenvolvimento e testes da aplicação • Docker Compose (`tools/docker-compose.yml`) para a API (porta 4000) e o frontend (porta 3000) • Comunicação entre serviços via `localhost` com portas dedicadas • Controle de versão integrado ao Git e ao GitHub • Lint, testes, *build* e verificação de segredos executados pelo CI em cada *pull request*
Cloud (Nuvem)	• Frontend hospedado na Vercel e API hospedada no Render como serviço web • Deploy feito por jobs do GitHub Actions, somente em *push* na `main` e após aprovação do CI • Verificação de prontidão da API após cada deploy e *health check* em `/api/health` • Banco de dados MongoDB Atlas em nuvem (cluster gerenciado) • Variáveis de ambiente gerenciadas nas plataformas e segredos mantidos nos *secrets* do GitHub
Dispositivos Móveis	• Interface responsiva (PWA) adaptada para smartphones e tablets • Service Worker para cache offline e notificações push • Compatibilidade com navegadores móveis modernos (Chrome, Safari) • Push Notifications via Web Push API (VAPID) • Suporte para iOS e Android via navegador
Segurança	• SSL/TLS fornecido automaticamente pela Vercel e pelo Render • Autenticação via JSON Web Tokens (JWT) com validação de `tokenVersion` • Senhas criptografadas com bcrypt • Dispositivos IoT identificados por chave (`X-Device-Key`) nas requisições à API • CORS configurado para origens permitidas • Controle de acesso por *roles* (`master`, `admin`, `membro`) • Verificação automática de segredos versionados no CI

7.4.2 Diagrama de Infraestrutura

Para ilustrar como os componentes se conectam:

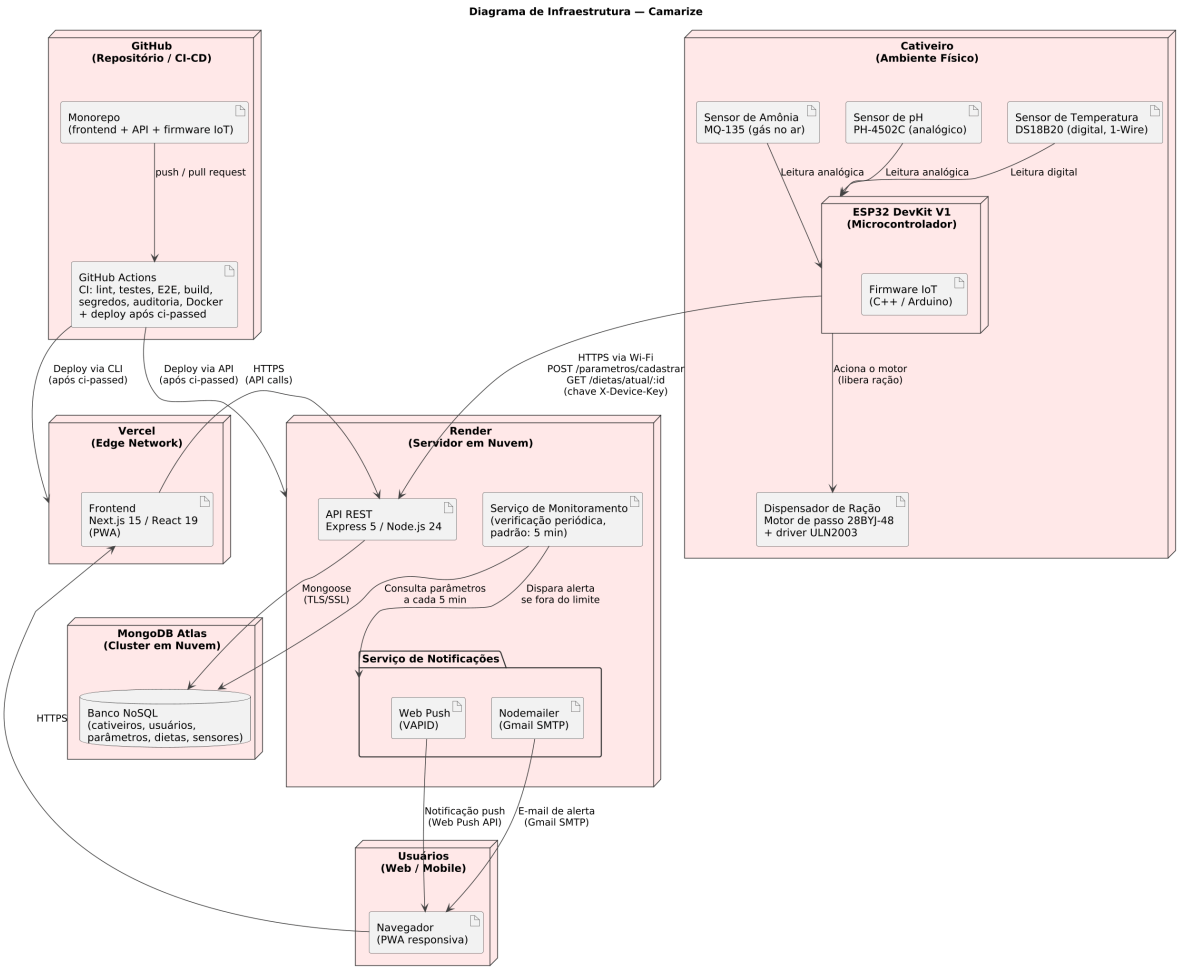


Figura 15: Diagrama de Infraestrutura do Sistema

- **Usuários Web/Mobile:** Acessam via navegador através de HTTPS (PWA responsiva)
- **Frontend:** Next.js 15 hospedado na Vercel
- **Backend:** API Express 5 hospedada no Render (Node.js 24)
- **Banco de Dados:** MongoDB Atlas (cluster em nuvem)
- **Monitoramento:** Serviço interno com verificação periódica dos parâmetros dos cativeiros
- **Notificações:** Nodemailer (Gmail SMTP) para alertas por e-mail e Web Push para notificações no navegador
- **ESP32:** Sensores IoT enviando dados de temperatura, pH e amônia via API REST

7.5 Arquitetura de Deploy

A arquitetura de deployment em produção utiliza serviços gerenciados na nuvem, com a publicação feita pelo GitHub Actions:

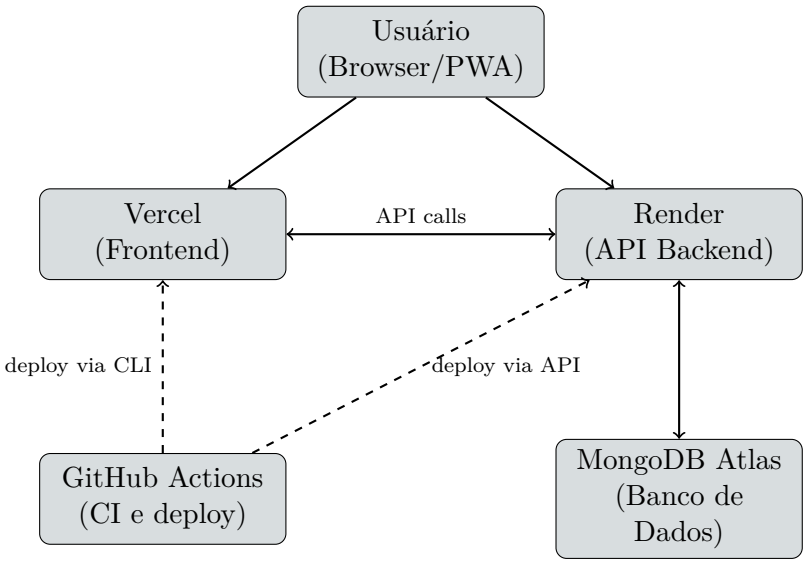


Figura 16: Arquitetura de Deployment em Produção

7.6 Configuração Docker Compose (Desenvolvimento)

Para o ambiente de desenvolvimento local, o arquivo `tools/docker-compose.yml` orquestra a API e o frontend. Em produção, o frontend é publicado na Vercel e a API no Render, e o CI valida o *build* das imagens Docker:

```
services:
  api:
    build: ../api
    env_file: ../api/.env.docker
    environment:
      - NODE_ENV=production
      - PORT=4000
    ports:
      - "4000:4000"
  frontend:
    build:
      context: ../front-react
      target: development
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:4000
      - NEXT_PUBLIC_SSE_URL=http://localhost:4000
    depends_on:
      - api
    ports:
      - "3000:3000"
```

7.7 Monitoramento e Alertas

O sistema possui um serviço de monitoramento automático que verifica periodicamente os parâmetros dos cativeiros:

- **Intervalo de Verificação:** Definido por variável de ambiente (`MONITORING_INTERVAL_MINUTES`, padrão: 5 minutos)

- **Parâmetros Monitorados:** Temperatura, pH e Amônia
- **Tolerâncias:** Temperatura (15%), pH (20%), Amônia (25%)
- **Severidade:** Classificação automática em `media` ou `alta`
- **Canais de Alerta:** E-mail (Gmail SMTP via Nodemailer) com respeito a quiet hours e rate limitingdia
- **Health Check:** Endpoint `/api/health` para verificação do status da API e banco de dados

7.8 Métricas e SLA

As métricas a seguir são **valores de referência** publicados pelas plataformas de hospedagem utilizadas (Vercel e Render), e não medições obtidas diretamente do projeto. Servem como parâmetros de expectativa para o comportamento da infraestrutura em produção:

- **Disponibilidade:** 99.9% (SLA garantido pela Vercel e Render em seus respectivos planos)
- **Tempo de Resposta:** $<$ 200ms (média estimada, favorecida pela edge network da Vercel)
- **Taxa de Erro:** $<$ 0.1% (referência da plataforma)
- **Throughput:** 1000 requisições/segundo (capacidade nominal da infraestrutura)
- **Deploy Frequency:** A cada push na branch `main` (deploy contínuo)
- **Lead Time:** $<$ 1 hora (estimativa baseada no fluxo atual de desenvolvimento)
- **MTTR:** $<$ 30 minutos (Mean Time To Recovery, estimativa operacional)

Conclusão

O desenvolvimento do projeto Camarize possibilitou a elaboração de uma solução tecnológica voltada ao monitoramento de parâmetros relacionados à criação de camarões em cativeiro, integrando conceitos de Internet das Coisas (IoT), desenvolvimento de software e armazenamento de dados em nuvem. Ao longo do projeto, foram desenvolvidos artefatos de documentação e modelagem, incluindo análises de negócio, diagramas UML, modelagem do banco de dados, arquitetura de infraestrutura, prototipação de interfaces e processos de integração e entrega contínua.

A implementação do protótipo permitiu integrar os componentes de hardware e software, estabelecendo a comunicação entre os sensores, a API e os serviços responsáveis pelo armazenamento e disponibilização das informações. Também foram desenvolvidos recursos de visualização e acompanhamento dos dados, além de mecanismos de notificação, conforme as funcionalidades previstas para o sistema.

Os testes realizados tiveram como finalidade verificar o funcionamento básico dos componentes e a integração entre as partes da solução, permitindo identificar limitações e oportunidades de melhoria. Entre os desafios encontrados, destacam-se as limitações dos sensores utilizados, a dependência de conectividade com a internet e a necessidade de aprimorar a validação das medições em condições reais de criação. Essas limitações devem ser consideradas na interpretação dos dados apresentados pelo sistema e em possíveis evoluções do projeto.

No âmbito do desenvolvimento e da manutenção do software, a utilização de ferramentas de versionamento, automação de testes, conteinerização e integração contínua contribuiu para organizar o processo de desenvolvimento e padronizar as verificações realizadas antes da disponibilização das aplicações. O pipeline de CI/CD foi configurado e executado com sucesso, demonstrando a viabilidade da automação adotada no projeto.

Dessa forma, considera-se que os objetivos propostos para o desenvolvimento do protótipo foram alcançados, demonstrando a viabilidade da integração entre hardware, software e serviços em nuvem para apoiar o monitoramento da criação de camarões em cativeiro. Embora a solução ainda apresente limitações que podem ser trabalhadas em versões futuras, os resultados obtidos constituem uma base para evoluções, como o aprimoramento da precisão das medições, a ampliação dos testes em ambientes reais e a inclusão de novos recursos de acompanhamento e análise.

Referências

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: Guia do Usuário*. 2. ed. Rio de Janeiro: Elsevier, 2005.

CHIAVENATO, I.; SAPIRO, A. *Planejamento Estratégico: da Intenção aos Resultados*. 4. ed. São Paulo: Atlas, 2020.

KIM, G. et al. *Manual de DevOps: Como Obter Agilidade, Confiabilidade e Segurança em Organizações Tecnológicas*. Rio de Janeiro: Alta Books, 2018.